

Workflow and architecture patterns for working together 2: Architectures

Jacob Archambault

June 2, 2026

Outline

1 Current challenges and their canonical solutions

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice

Outline

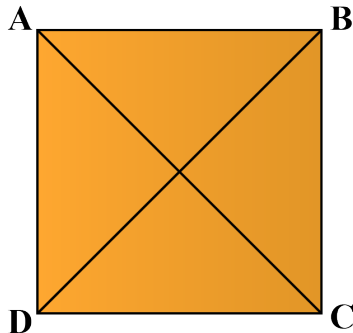
- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

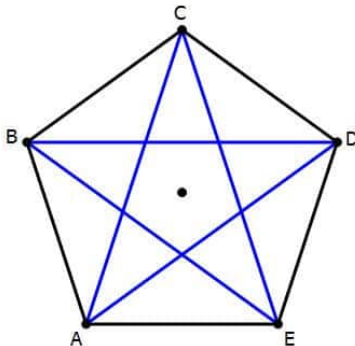
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals= $n(n-1)/2$



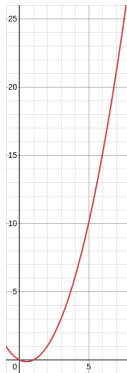
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals= $n(n-1)/2$



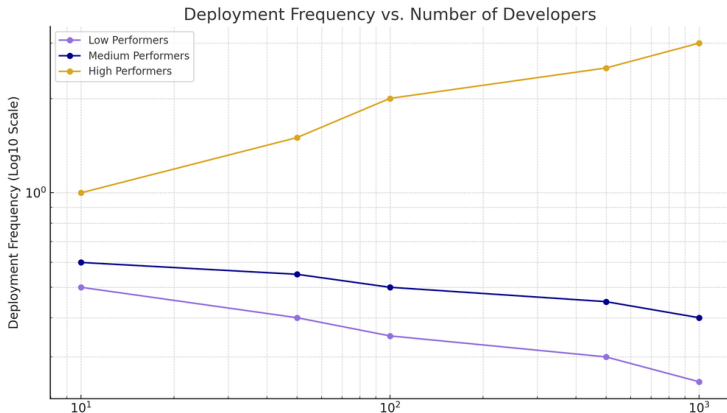
1975: The Mythical Man Month, Fred Brooks

- number of direct communication paths for n individuals=
$$n(n-1)/2$$



The Mythical Man Month, Fred Brooks (cont.)

- corollary: adding more people to a project can lead not only to diminishing returns on delivery speed, but to objectively less work being completed



Challenges

- multiple teams working in the same repository

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk
 - risk of cross-project interference

Challenges

- multiple teams working in the same repository
- lack of direct collaboration, and occasionally trust, between development, operations, and testing
- late integration of contributor code
 - feedback delays
 - merge conflict regressions
 - sunk costs fallacy risks
- coupling of all services at the database layer
 - high storage costs
 - single point of failure risk
 - risk of cross-project interference

Canonical solutions to these challenges

- multiple teams $\rightarrow$ inverse Conway maneuver

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

Canonical solutions to these challenges

- multiple teams → inverse Conway maneuver
- collaboration → business stream-aligned teams over activity oriented teams
- late integration → adopting a true continuous integration workflow
- database coupling → Bezos API mandate/“Microservices”

1967: Conway's law

'Organizations which design systems [...] are constrained to produce designs which are copies of the communication structures of these organizations.' - Melvin Conway, 'How do Committees Invent?' *Datamation*, 1967

Conway's law: examples

- separate .csproj projects for tests written by QA
- separate development and config repositories for the same transaction signifying low communication between development and operations
- all business documentation in the same repository, separate from the repositories that the documentation is for

Conway's law: corollaries

- Conway's law can't be fought

Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team

Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
 - separate the codebases for improved velocity

Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
 - separate the codebases for improved velocity
 - join the teams for improved code sharing and communication (Inverse conway maneuver)

Conway's law: corollaries

- Conway's law can't be fought
- Teams working in the same part of the same codebase at the same time are the same team
 - separate the codebases for improved velocity
 - join the teams for improved code sharing and communication (Inverse conway maneuver)
 - branching-based strategies for team management compromise between the above approaches without their benefits

Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.

Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
 - pull requests

Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
 - pull requests
 - testing

Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
 - pull requests
 - testing
 - deployment

Revisiting Conway's law: defining a team

- A *team* is the minimal socio-technical unit that can deliver a production change with no additional coordination beyond mandated governance or custodial controls.
 - pull requests
 - testing
 - deployment

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
 - $P(\text{no conflicts}) < 10\%$ where $n \geq 4$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
 - $P(\text{no conflicts}) < 10\%$ where $n \geq 4$
 - $P(\text{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for merge consistency: isolated commits

- $C = \{a, b, c\}$
- $\mathcal{P}(C) = \{\{\emptyset\}, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}$
- $P(\text{no conflicts}) = \mathcal{P}(\emptyset) = \frac{1}{8} = 12.5\%$
- $P(\text{no conflicts}) = \frac{1}{2^n}$ where $|C| = n$
 - $P(\text{no conflicts}) < 10\%$ where $n \geq 4$
 - $P(\text{no conflicts}) < 1\%$ where $n \geq 7$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A)) =$ probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
 - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
 - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
 - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets
 - $= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)}$ where $n = |A|$ and $n_i = |A_i|$
 - $= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$
 - $= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

$$\begin{aligned} \bullet &= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)} \text{ where } n = |A| \text{ and } n_i = |A_i| \\ \bullet &= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3} \\ \bullet &= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3} \\ \bullet &= \frac{1}{32 - 2 - 4 - 4 + 3} \end{aligned}$$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

$$\begin{aligned} \bullet &= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)} \text{ where } n = |A| \text{ and } n_i = |A_i| \\ \bullet &= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3} \\ \bullet &= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3} \\ \bullet &= \frac{1}{32 - 2 - 4 - 4 + 3} \\ \bullet &= \frac{1}{32 - 7} \end{aligned}$$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

$$\begin{aligned} \bullet &= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)} \text{ where } n = |A| \text{ and } n_i = |A_i| \\ \bullet &= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3} \\ \bullet &= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3} \\ \bullet &= \frac{1}{32 - 2 - 4 - 4 + 3} \\ \bullet &= \frac{1}{32 - 7} \\ \bullet &= \frac{1}{25} \end{aligned}$$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

$$\begin{aligned}
 \bullet &= \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)} \text{ where } n = |A| \text{ and } n_i = |A_i| \\
 \bullet &= \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3} \\
 \bullet &= \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3} \\
 \bullet &= \frac{1}{32 - 2 - 4 - 4 + 3} \\
 \bullet &= \frac{1}{32 - 7} \\
 \bullet &= \frac{1}{25} \\
 \bullet &= 4\%
 \end{aligned}$$

A naive probability theory for merge consistency: modeling branches as sets of commits

- $A = A_1 \sqcup A_2 \sqcup A_3$ where $A_1 = \{a\}$, $A_2 = \{b, c\}$, $A_3 = \{d, e\}$
- Assume $A_1 \cdots A_3$ are known good sets of commits. Then $P(\text{no conflicts}(A))$ = probability of selecting the null set from the set containing both the null set and all remaining unchecked potential conflict sets

$$\bullet = \frac{1}{2^n - (2^{n_1} - 1) - (2^{n_2} - 1) - (2^{n_3} - 1)} \text{ where } n = |A| \text{ and } n_i = |A_i|$$

$$\bullet = \frac{1}{2^n - 2^{n_1} - 2^{n_2} - 2^{n_3} + 3}$$

$$\bullet = \frac{1}{2^5 - 2^1 - 2^2 - 2^2 + 3}$$

$$\bullet = \frac{1}{32 - 2 - 4 - 4 + 3}$$

$$\bullet = \frac{1}{32 - 7}$$

$$\bullet = \frac{1}{25}$$

$$\bullet = 4\%$$

A naive probability theory for merge consistency: general definition for distinct branches diverging from a common ancestor

- Let A be the set of unmerged commits across k distinct branches $A_1 \cdots A_k$, where $A = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_k$, $n = |A|$, and $n_i = |A_i|$ for $1 \leq i \leq k$. Then

A naive probability theory for merge consistency: general definition for distinct branches diverging from a common ancestor

- Let A be the set of unmerged commits across k distinct branches $A_1 \cdots A_k$, where $A = A_1 \sqcup A_2 \sqcup \cdots \sqcup A_k$, $n = |A|$, and $n_i = |A_i|$ for $1 \leq i \leq k$. Then

$$P(\text{no conflicts}) = \frac{1}{2^n - (\sum_{i=1}^k 2^{n_i}) + k}$$

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 **Branching: theory and practice**
 - Theory
 - **Practice**
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

The State of DevOps Report: throughput metrics

- lead time for changes

The State of DevOps Report: throughput metrics

- lead time for changes
- deployment frequency

The State of DevOps Report: stability metrics

- change failure rate

The State of DevOps Report: stability metrics

- change failure rate
- mean time to restore

The State of DevOps Report: results

'Astonishingly, these results demonstrate that there is no trade-off between improving performance and achieving higher levels of stability and quality. Rather, high performers do better at all of these measures. This is precisely what the Agile and Lean movements predict, but much dogma in our industry still rests on the false assumption that moving faster means trading off against other performance goals, rather than enabling and reinforcing them.'
- Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate* (2017), p. 14

The State of DevOps Report: results

When compared to low performers, elite performers realize

127x

faster lead
time

182x

more
deployments
per year

8x

lower change
failure rate

2293x

faster failed
deployment
recovery times

- 2024 State of DevOps Report

Approximating DevOps metrics in our current environment

- deployment frequency → GitHub Actions run count

Approximating DevOps metrics in our current environment

- deployment frequency → GitHub Actions run count
- change failure rate → GitHub Actions pipeline failure rate

Approximating DevOps metrics in our current environment

- deployment frequency → GitHub Actions run count
- change failure rate → GitHub Actions pipeline failure rate
- mean time to restore → percentage of merge commits to value-adding commits

Approximating DevOps metrics in our current environment

- deployment frequency → GitHub Actions run count
- change failure rate → GitHub Actions pipeline failure rate
- mean time to restore → percentage of merge commits to value-adding commits

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr
- 29% better velocity than all other contributions combined since 23 Mar

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr
- 29% better velocity than all other contributions combined since 23 Mar
- 60% the velocity of all other initiatives combined since 21 Jan

Throughput: overall velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 18% better velocity than all other contributions combined since 15 Apr
- 29% better velocity than all other contributions combined since 23 Mar
- 60% the velocity of all other initiatives combined since 21 Jan

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 136% better per contributor velocity than all other contributions combined since 15 Apr

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 136% better per contributor velocity than all other contributions combined since 15 Apr
- 158% better per contributor velocity than all other contributions combined since 23 Mar

Throughput: per contributor velocity metrics

Branch	Runs	Earliest run	Contributors
NG_Claims_837D_COB	74	23 Mar	5
PEDI-35120_837D_COB	85	15 Apr	5
Either COB branch	159	23 Mar	6
All other branches	123	23 Mar	12
	72	15 Apr	10
	266	21 Jan	

Source: GH Actions Simulation tests, 28 May 2026

- 136% better per contributor velocity than all other contributions combined since 15 Apr
- 158% better per contributor velocity than all other contributions combined since 23 Mar

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar
- 150% better simulation test pass rate since 15 Apr

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar
- 150% better simulation test pass rate since 15 Apr
- 158% better unit test pass rate since 15 Apr

Stability: pipeline failure rate metrics

Branch	Simulation test failures	Unit test failures	Earliest commit
NG_Claims_837D_COB	56/74 (76%)	56/74 (76%)	23 Mar
PEDI-35120_837D_COB	20/85 (23.5%)	21/85 (24.7%)	15 Apr
Either COB branch	76/159 (48%)	77/159 (48%)	23 Mar
All extant branches	73/123 (59%)	87/123 (71%)	23 Mar
	50/72 (69%)	51/72 (71%)	15 Apr
	92/161 (57%)	104/161 (64%)	21 Jan

Source: GH Actions, 28 May 2026

- 28% better simulation test pass rate since 23 Mar
- 76% better unit test pass rate since 23 Mar
- 150% better simulation test pass rate since 15 Apr
- 158% better unit test pass rate since 15 Apr

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

- 50% less merge work

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

- 50% less merge work
- 25% less backmerge work

Stability metrics: merge work ratio

Branch	Merge commit % since 15 Apr
PEDI-35120_837D_COB	12/91 (13%)
All other branches	23/88 (26%)
Excluding PR merges	14/79 (18%)

- 50% less merge work
- 25% less backmerge work
- Source command:

```
git log --since="2026-04-15"  
--oneline <--merges> <--all>  
<^PEDI-35120_837D_COB> | <grep -iv 'merge pull  
request'> | wc -l
```

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

Team composition

- All individuals working in same git repository are in the same daily scrum

Team composition

- All individuals working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I can set this up.

Team composition

- All individuals working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I can set this up.
- teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)

Team composition

- All individuals working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I can set this up.
- teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- these teams can move work forward without waiting on or requiring outside assistance from another team

Team composition

- All individuals working in same git repository are in the same daily scrum
- Each team in the above sense has its own Microsoft Teams chat with only its own contributors in it. I can set this up.
- teams are *stream-aligned*, i.e., they focus on a particular business area (in our case, either a transaction type or a closely related set of transaction types)
- these teams can move work forward without waiting on or requiring outside assistance from another team

Continuous integration: Enablers

- improving CI pipeline runtime

Continuous integration: Enablers

- improving CI pipeline runtime
- team discipline

Continuous integration: Enablers

- improving CI pipeline runtime
- team discipline
- fast PR turnaround

Continuous integration: Enablers

- improving CI pipeline runtime
- team discipline
- fast PR turnaround

Team composition: enabling teams

- Enabling team - helps a Stream-aligned team to overcome obstacles. Also detects missing capabilities.

Team composition: enabling teams

- Enabling team - helps a Stream-aligned team to overcome obstacles. Also detects missing capabilities.
- The 837 COB team will serve as an *enabling team* for piloting a teams-based branching strategy

Team composition: enabling teams

- Enabling team - helps a Stream-aligned team to overcome obstacles. Also detects missing capabilities.
- The 837 COB team will serve as an *enabling team* for piloting a teams-based branching strategy
- Strategy will be rolled out first for claims, then reimbursements, then other transactions

Team composition: enabling teams

- Enabling team - helps a Stream-aligned team to overcome obstacles. Also detects missing capabilities.
- The 837 COB team will serve as an *enabling team* for piloting a teams-based branching strategy
- Strategy will be rolled out first for claims, then reimbursements, then other transactions

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 **Going forward**
 - Team Composition
 - **Branching and committing**
 - Testing
 - Pull Requests
 - Pipelines
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

Branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch

Branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- Every commit is pushed to origin only and immediately after passing local testing

Branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- Every commit is pushed to origin only and immediately after passing local testing
- Every git commit message begins with a Jira ticket number

Branching and committing

- All commit work done by a given team, by both developers and testers, is committed directly to the same shared branch
- Every commit is pushed to origin only and immediately after passing local testing
- Every git commit message begins with a Jira ticket number

Testing

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 **Going forward**
 - Team Composition
 - Branching and committing
 - **Testing**
 - Pull Requests
 - Pipelines
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- Every change is tested by the test suite locally before committing

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.
- Testing need not be end-to-end, but may simply test a component

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.
- Testing need not be end-to-end, but may simply test a component
- Goal is to move most testing into the test suite so that manual checks become minimal.

- Every change is tested by the test suite locally before committing
- Tests are written against new code as that code is being written
- We don't push up non-passing tests. Instead, we find the person we need to talk to and work out a passing solution, then push up.
- Testing need not be end-to-end, but may simply test a component
- Goal is to move most testing into the test suite so that manual checks become minimal.

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 **Going forward**
 - Team Composition
 - Branching and committing
 - Testing
 - **Pull Requests**
 - Pipelines
- 5 **Architecture**
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- daily PRs - stacked PRs

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 **Going forward**
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - **Pipelines**
- 5 Architecture
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

- pipeline is definitive for deployment - daily stale branch job

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 **Architecture**
 - Techniques for integrating partially complete work
 - Clean code
 - On the way to Microservices

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 **Architecture**
 - Techniques for integrating partially complete work
 - **Clean code**
 - On the way to Microservices

Outline

- 1 Current challenges and their canonical solutions
- 2 Conway's law
- 3 Branching: theory and practice
 - Theory
 - Practice
- 4 Going forward
 - Team Composition
 - Branching and committing
 - Testing
 - Pull Requests
 - Pipelines
- 5 **Architecture**
 - Techniques for integrating partially complete work
 - Clean code
 - **On the way to Microservices**